

# LC 2685 → Count the number of Complete Components

Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 30, 2026

## Concept & Notes

a connected component is called complete if every node is connected to every other node

this question has 2 parts

→ 1) find every connected component

→ 2) verify its complete

for a graph to be complete → let  $k = 4$

then every node connects with 3 others

hence total degree =  $(4 \times 3) = 12$

every edge counted twice so  $12 / 2 = 6$

lets generalize

k nodes connect with (k-1) nodes

so edges =  $[k \times (k-1)] / 2$  → condition of completeness

hence while performing DFS

→ we collect number of nodes for k

→ sum of degrees for verification to see if it equals  $[k \times (k-1)] / 2$

## Solution 1: DFS Approach

---

```

class Solution {
public:
    void dfs(vector<vector<int>>& adj, vector<int>& seen, int curr, int& k, int& degsum) {
        seen[curr] = 1;
        k++;
        degsum += adj[curr].size();

        for(auto &it : adj[curr]) {
            if(!seen[it]) {
                dfs(adj, seen, it, k, degsum);
            }
        }
    }

    int countCompleteComponents(int n, vector<vector<int>>& edges) {
        vector<vector<int>> adj(n);
        for(auto &it : edges) {
            int u = it[0];
            int v = it[1];
            adj[u].push_back(v);
            adj[v].push_back(u);
        }

        //do parrrt me divide krna padega
        //ek to complete components pe dfs krenge

        //k nodes ko k-1 nodes se connect krna he to edges=(k*k-1)/(2)
        //to jab dfs krenge to us ek connected compoent me jitne nodes honge wahi k hoga
        //saath hi saath degree ka sum nikaal lenge , usko divide by 2 will be actual edges
        //agar dono equal he to ans++

        int ans = 0;
        vector<int> seen(n, 0);

        for(int i = 0; i < n; i++) {
            if(!seen[i]) {
                int k = 0;
                int degsum = 0;
                dfs(adj, seen, i, k, degsum);

                if(degsum == (k * (k - 1))) ans++; //degsum/2==((k*(k-1))/2 , 2 se 2 c
                //teri bandi banna cancel
                //bachpana nhi jaara
            }
        }
        return ans;
    }
};

```

## Solution 2: BFS Approach

---

```

class Solution {
public:
    int countCompleteComponents(int n, vector<vector<int>>& edges) {
        vector<vector<int>> adj(n);
        for(auto &it : edges) {
            int u = it[0];
            int v = it[1];
            adj[u].push_back(v);
            adj[v].push_back(u);
        }

        //do parrt me divide krna padega
        //ek to complete components pe bfs krenge

        //k nodes ko k-1 nodes se connect krna he to edges=(k*k-1)/(2)
        //to jab bfs krenge to us ek connected compoent me jitne nodes honge wahi k hoga
        //saath hi saath degree ka sum nikaal lenge , usko divide by 2 will be actual edges
        //agar dono equal he to ans++

        int ans = 0;
        vector<int> seen(n, 0);

        for(int i = 0; i < n; i++) {
            if(!seen[i]) {
                int k = 0;
                int degsum = 0;
                queue<int> q;
                q.push(i);
                seen[i] = 1;

                while(!q.empty()) {
                    int curr = q.front();
                    q.pop();
                    k++;
                    degsum += adj[curr].size();

                    for(auto &it : adj[curr]) {
                        if(!seen[it]) {
                            seen[it] = 1;
                            q.push(it);
                        }
                    }
                }

                if(degsum == (k * (k - 1))) ans++; //degsum/2==((k*(k-1))/2 , 2 se 2 cancel
                                                    //teri bandi banna cancel
                                                    //bachpana nhi jaara
            }
        }
        return ans;
    }
};

```

```
}  
};
```